CONSTRUCTION MATERIAL MANAGEMENT SYSTEM

CONTENT

1. Overview
2. System Architecture
3. Requirements
4. ER Diagram
5. Relational Schema
6. Normalization
7. Data Dictionary
8. Database Objects
9. Application Workflow and API Endpoints
10. Test Cases
11. Setup and Deployment
12. Maintenance Guide
13. Limitations and Future Enhancements

1.OVERVIEW

BuildStock is a Construction Material Management System designed to manage construction materials, suppliers, purchases, inventory, and material allocations across construction sites. It provides a centralized MySQL database and a web-based interface to replace manual registers and spreadsheets.

The system enables users to maintain material records, track stock quantities, record purchases, allocate materials to construction sites, identify low-stock materials, and generate reports on purchases and material usage.

The application uses Node.js and Express.js for backend processing, HTML, CSS, and JavaScript for the frontend, and MySQL for data storage and database-level business logic.

1.1. USER ROLES

Store Manager: Maintains materials, monitors inventory, checks low-stock warnings, allocates materials to construction sites, and views reports.

Purchase Officer: Maintains supplier information and records material purchases.

Site Engineer / Supervisor: Receives materials allocated to construction sites and monitors material usage.

Administrator: Maintains employee records and construction site information.

These roles describe the intended responsibilities of users. The current application does not implement login functionality or role-based access control.

1.2. KEY FEATURES

Material Management: Add, view, edit, delete, and search construction material records.

Supplier Management: Maintain supplier details, including contact information and location.

Employee Management: Maintain employee records and their assigned roles.

Construction Site Management: Maintain site names, locations, statuses, and managers.

Purchase Management: Record purchases containing multiple materials and calculate purchase totals automatically.

Inventory Management: Track available stock and update quantities automatically when materials are purchased or allocated.

Material Allocation: Record material issued to construction sites and prevent allocations that exceed available stock.

Low-Stock Monitoring: Identify materials whose quantities have reached or fallen below their minimum stock levels.

Reporting: Display inventory values, purchase summaries, supplier purchase costs, and site-wise material usage costs.

Database Integrity: Use constraints, foreign keys, triggers, functions, views, stored procedures, and transactions to maintain data consistency.

2. SYSTEM ARCHITECTURE

BuildStock follows a three-layer architecture consisting of the presentation layer, application layer, and database layer.

2.1. PRESENTATION LAYER

Files: public/index.html, public/style.css, public/app.js

The presentation layer provides the user interface. It contains navigation options, forms, tables, dashboard information, reports, and success or error messages.

JavaScript uses the Fetch API to send HTTP requests to the backend and display the returned data.

2.2. APPLICATION LAYER

File: server.js

The application layer is developed using Node.js and Express.js. It receives requests from the frontend, validates input, executes SQL queries through the mysql2 library, manages transactions, and returns JSON responses.

It also handles application errors and communicates database errors to the frontend in a readable format.

2.3. DATABASE LAYER

Database: MySQL 8.0+

The database stores application records and enforces business rules using relational tables, primary keys, foreign keys, constraints, views, a stored function, a stored procedure, and triggers.

2.4. SYSTEM DATA FLOW

Browser → Node.js and Express.js → MySQL Database

The browser sends HTTP requests to the Express server. The server executes the required SQL operations against the MySQL database and receives the results. It then returns JSON responses to the browser, where the interface displays the information.

The browser does not communicate directly with MySQL. All database operations are handled through the backend API.

2.5. ARCHITECTURE DIAGRAM

The following diagram represents the communication between the application layers.

Browser (HTML, CSS, JavaScript)

↓

HTTP Requests and JSON Responses

↓

Node.js and Express.js (server.js)

↓

SQL Queries through mysql2

↓

MySQL Database (buildstock)

The database returns query results to the backend, which sends the appropriate response to the browser.

3. REQUIREMENTS

3.1. FUNCTIONAL REQUIREMENTS

FR1: The system shall allow users to add, view, edit, delete, and search materials, including their minimum stock levels.

FR2: The system shall allow users to maintain supplier records.

FR3: The system shall maintain employee records and construction site details, including site location, status, and manager.

FR4: The system shall allow users to record purchases containing multiple material items.

FR5: The system shall calculate purchase totals and increase stock quantities automatically when purchase items are recorded.

FR6: The system shall allow users to allocate materials to construction sites and reduce available stock automatically.

FR7: The system shall reject allocations exceeding the available stock quantity.

FR8: The system shall display inventory quantities, stock status, and total stock value.

FR9: The system shall display a dashboard containing key counts and low-stock warnings.

FR10: The system shall identify materials at or below their minimum stock levels and calculate the stock shortfall.

FR11: The system shall display material usage and associated costs for a selected construction site.

FR12: The system shall provide reports showing purchase costs by supplier and material usage costs by site.

3.2. NON-FUNCTIONAL REQUIREMENTS

Data Integrity: Primary keys, foreign keys, and constraints shall maintain valid relationships between database records.

Reliability: Transactions shall ensure that purchases and allocations are completed successfully or rolled back when an operation fails.

Usability: The interface shall provide clear navigation, understandable messages, and organized information.

Maintainability: Generic CRUD routines shall reduce code duplication when managing common database tables.

Consistency: Database triggers shall automatically update stock quantities and purchase totals when the corresponding records are inserted.

3.3. SOFTWARE REQUIREMENTS

Operating System: Windows 10 or later, macOS, or Linux.

Database Management System: MySQL 8.0 or later.

Runtime Environment: Node.js 18 or later.

Backend Framework: Express.js.

Database Driver: mysql2.

Frontend Technologies: HTML, CSS, and JavaScript.

Web Browser: Any modern web browser.

Development Tools: Visual Studio Code and MySQL Workbench.

3.4. HARDWARE REQUIREMENTS

Minimum RAM: 4 GB.

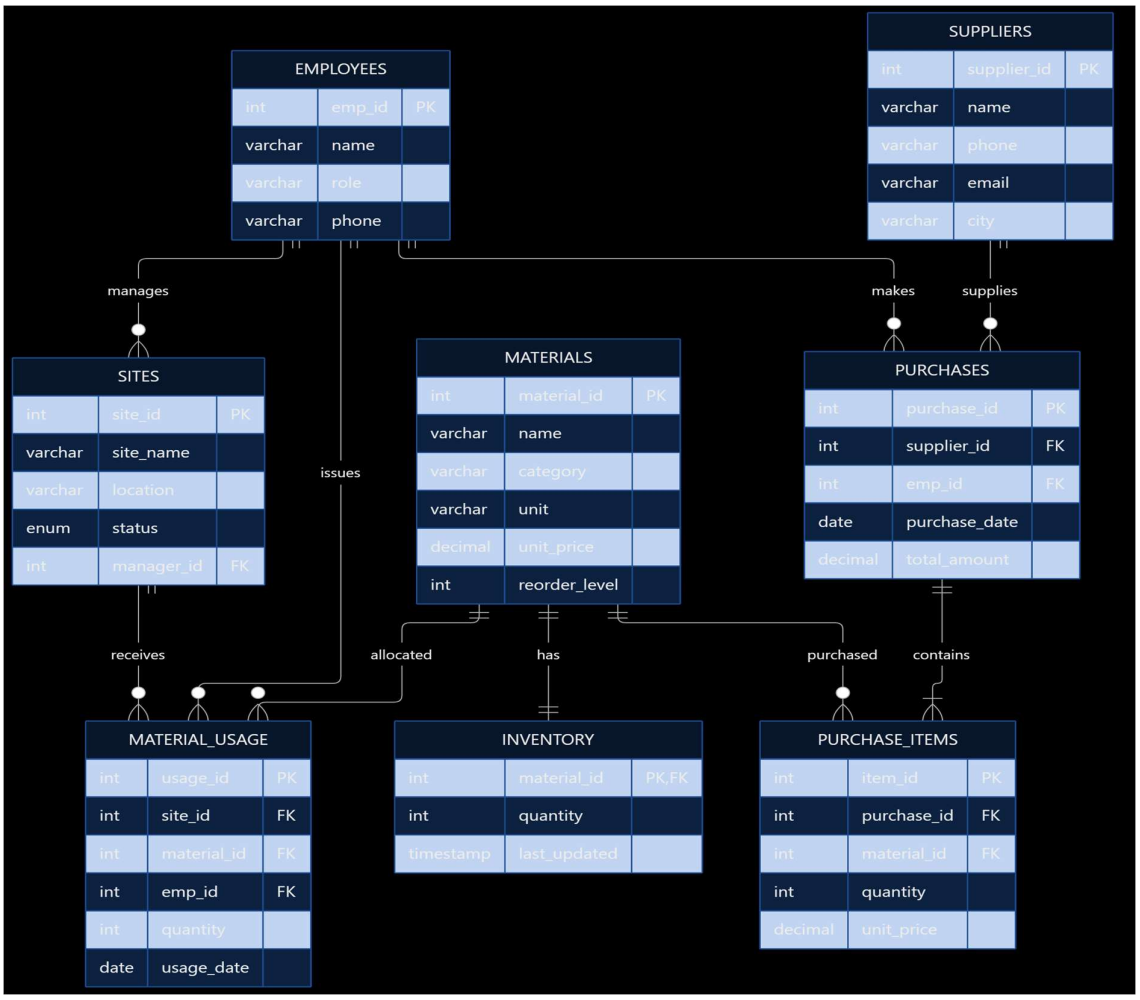
Available Disk Space: 1 GB.

Processor: Any processor capable of running the supported operating system and Node.js.

Network: Local network access is not required for a local installation, but the browser must be able to access the running application.

4. ER DIAGRAM

The Entity-Relationship (ER) diagram represents the entities, attributes, and relationships used in the BuildStock database.



4.1. ENTITIES AND ATTRIBUTES

EMPLOYEES

Primary Key: emp_id

Attributes: name, role, phone.

SUPPLIERS

Primary Key: supplier_id

Attributes: name, phone, email, city.

MATERIALS

Primary Key: material_id

Attributes: name, category, unit, unit_price, reorder_level.

SITES

Primary Key: site_id

Attributes: site_name, location, status, manager_id.

PURCHASES

Primary Key: purchase_id

Attributes: supplier_id, emp_id, purchase_date, total_amount.

PURCHASE_ITEMS

Primary Key: item_id

Attributes: purchase_id, material_id, quantity, unit_price.

INVENTORY

Primary Key: material_id

Attributes: quantity, last_updated.

MATERIAL_USAGE

Primary Key: usage_id

Attributes: site_id, material_id, emp_id, quantity, usage_date.

4.2. ENTITY RELATIONSHIPS

Employees and Sites: One employee can manage multiple sites. Each site can have one assigned manager or no manager.

Employees and Purchases: One employee can record multiple purchases.

Employees and Material Usage: One employee can issue materials multiple times.

Suppliers and Purchases: One supplier can supply multiple purchases.

Purchases and Purchase Items: One purchase contains one or more purchase items.

Materials and Purchase Items: One material can appear in multiple purchase items.

Materials and Inventory: Each material has exactly one inventory record.

Sites and Material Usage: One site can have multiple material usage records.

Materials and Material Usage: One material can be allocated to multiple sites through material usage records.

code can be used to generate the ER diagram in a compatible Mermaid editor.

5. RELATIONAL SCHEMA

The relational schema defines the database tables, their attributes, primary keys, and foreign key relationships.

5.1. TABLE DEFINITIONS

Employees (emp_id PK, name, role, phone)

Suppliers (supplier_id PK, name, phone, email, city)

Materials (material_id PK, name UNIQUE, category, unit, unit_price, reorder_level)

Sites (site_id PK, site_name, location, status, manager_id FK → Employees.emp_id)

Purchases (purchase_id PK, supplier_id FK → Suppliers.supplier_id, emp_id FK → Employees.emp_id, purchase_date, total_amount)

Purchase_Items (item_id PK, purchase_id FK → Purchases.purchase_id, material_id FK → Materials.material_id, quantity, unit_price)

Inventory (material_id PK and FK → Materials.material_id, quantity, last_updated)

Material_Usage (usage_id PK, site_id FK → Sites.site_id, material_id FK → Materials.material_id, emp_id FK → Employees.emp_id, quantity, usage_date)

5.2. PRIMARY KEYS

Primary keys uniquely identify each record in a table.

The primary keys are emp_id, supplier_id, material_id, site_id, purchase_id, item_id, material_id in Inventory, and usage_id in Material_Usage.

5.3. FOREIGN KEYS

Foreign keys maintain relationships between tables.

Sites.manager_id references Employees.emp_id.

Purchases.supplier_id references Suppliers.supplier_id.

Purchases.emp_id references Employees.emp_id.

Purchase_Items.purchase_id references Purchases.purchase_id.

Purchase_Items.material_id references Materials.material_id.

Inventory.material_id references Materials.material_id.

Material_Usage.site_id references Sites.site_id.

Material_Usage.material_id references Materials.material_id.

Material_Usage.emp_id references Employees.emp_id.

5.4. REFERENTIAL ACTIONS

Sites.manager_id: ON DELETE SET NULL. When a referenced employee is deleted, the manager reference is set to NULL.

Purchase_Items.purchase_id: ON DELETE CASCADE. Deleting a purchase also deletes its associated purchase items.

Inventory.material_id: ON DELETE CASCADE. Deleting a material also deletes its corresponding inventory record.

Other foreign keys: Use RESTRICT or the default restrictive behavior unless another action

6. NORMALIZATION

Normalization organizes database tables to minimize unnecessary duplication and prevent data inconsistencies.

6.1. UNNORMALIZED FORM (UNF)

A single purchase record containing supplier details, employee details, and multiple material entries can result in repeating groups and duplicated information.

For example, if a purchase contains cement, steel rods, and bricks, the supplier and employee details may be repeated for every material.

6.2. FIRST NORMAL FORM (1NF)

First Normal Form requires each attribute to contain an atomic value and removes repeating groups.

In BuildStock, each purchased material is represented by an individual row in Purchase_Items. Each row contains one purchase reference, one material reference, a quantity, and a unit price.

6.3. SECOND NORMAL FORM (2NF)

Second Normal Form requires a table to be in 1NF and eliminates partial dependencies on a composite key.

Purchase-level attributes, such as purchase date, supplier, employee, and total amount, are maintained in Purchases.

Material-level attributes, such as name, category, unit, and catalogue price, are maintained in Materials.

The quantity purchased and the price paid for each item are maintained in Purchase_Items.

This separation ensures that information is stored in the table most closely associated with its key.

6.4. THIRD NORMAL FORM (3NF)

Third Normal Form requires a table to be in 2NF and eliminates transitive dependencies.

Supplier information, such as phone number, email, and city, is maintained in Suppliers rather than repeated in Purchases.

Employee information, such as name and role, is maintained in Employees.

Current stock quantities are maintained separately in Inventory because they change independently of the material's descriptive information.

This separation reduces duplication and improves data consistency.

6.5. BOYCE-CODD NORMAL FORM (BCNF)

BCNF requires every determinant in a relation to be a candidate key.

The schema is intended to satisfy BCNF under the assumed functional dependencies. Formal confirmation requires checking the functional dependencies of each relation.

6.6. CONTROLLED REDUNDANCY

The total_amount attribute in Purchases stores the sum of its associated purchase item values. This value could be calculated from Purchase_Items, but storing it simplifies reporting.

The trg_purchase_stock trigger updates the purchase total when purchase items are inserted.

This is controlled redundancy. The trigger and any future update or deletion operations must preserve consistency between the stored total and the purchase item records.

7. DATA DICTIONARY

The data dictionary describes each table's columns, data types, constraints, and purposes.

7.1. EMPLOYEES TABLE

Column: emp_id

Data Type: INT

Constraints: PRIMARY KEY, AUTO_INCREMENT.

Description: Unique identifier for each employee.

Column: name

Data Type: VARCHAR(80)

Constraints: NOT NULL.

Description: Full name of the employee.

Column: role

Data Type: VARCHAR(50)

Constraints: NOT NULL.

Description: Employee's role, such as Store Manager or Site Engineer.

Column: phone

Data Type: VARCHAR(15)

Constraints: Optional.

Description: Contact number of the employee.

7.2. SUPPLIERS TABLE

Column: supplier_id

Data Type: INT

Constraints: PRIMARY KEY, AUTO_INCREMENT.

Description: Unique identifier for each supplier.

Column: name

Data Type: VARCHAR(100)

Constraints: NOT NULL.

Description: Name of the supplier.

Column: phone

Data Type: VARCHAR(15)

Constraints: Optional.

Description: Supplier's contact number.

Column: email

Data Type: VARCHAR(100)

Constraints: Optional.

Description: Supplier's email address.

Column: city

Data Type: VARCHAR(50)

Constraints: Optional.

Description: City where the supplier is located.

7.3. MATERIALS TABLE

Column: material_id

Data Type: INT

Constraints: PRIMARY KEY, AUTO_INCREMENT.

Description: Unique identifier for each material.

Column: name

Data Type: VARCHAR(80)

Constraints: NOT NULL, UNIQUE.

Description: Name of the construction material.

Column: category

Data Type: VARCHAR(50)

Constraints: NOT NULL.

Description: Category of the material, such as Binding, Steel, or Masonry.

Column: unit

Data Type: VARCHAR(20)

Constraints: NOT NULL.

Description: Measurement unit, such as bag, ton, or piece.

Column: unit_price

Data Type: DECIMAL(10,2)

Constraints: NOT NULL, CHECK (unit_price > 0).

Description: Catalogue price per unit.

Column: reorder_level

Data Type: INT

Constraints: NOT NULL, DEFAULT 10.

Description: Minimum stock level used to identify low-stock materials.

7.4. SITES TABLE

Column: site_id

Data Type: INT

Constraints: PRIMARY KEY, AUTO_INCREMENT.

Description: Unique identifier for each construction site.

Column: site_name

Data Type: VARCHAR(100)

Constraints: NOT NULL.

Description: Name of the construction site.

Column: location

Data Type: VARCHAR(100)

Constraints: Optional.

Description: Physical location of the construction site.

Column: status

Data Type: ENUM('Active', 'On Hold', 'Completed')

Constraints: NOT NULL, DEFAULT 'Active'.

Description: Current status of the construction site.

Column: manager_id

Data Type: INT

Constraints: Foreign key referencing Employees.emp_id, ON DELETE SET NULL.

Description: Employee assigned to manage the construction site.

7.5. PURCHASES TABLE

Column: purchase_id

Data Type: INT

Constraints: PRIMARY KEY, AUTO_INCREMENT.

Description: Unique identifier for each purchase.

Column: supplier_id

Data Type: INT

Constraints: NOT NULL, FOREIGN KEY.

Description: Supplier associated with the purchase.

Column: emp_id

Data Type: INT

Constraints: NOT NULL, FOREIGN KEY.

Description: Employee who recorded the purchase.

Column: purchase_date

Data Type: DATE

Constraints: NOT NULL, DEFAULT current date.

Description: Date on which the purchase was recorded.

Column: total_amount

Data Type: DECIMAL(12,2)

Constraints: NOT NULL, DEFAULT 0.

Description: Total purchase value maintained through database trigger logic.

7.6. PURCHASE_ITEMS TABLE

Column: item_id

Data Type: INT

Constraints: PRIMARY KEY, AUTO_INCREMENT.

Description: Unique identifier for each purchase item.

Column: purchase_id

Data Type: INT

Constraints: NOT NULL, FOREIGN KEY, ON DELETE CASCADE.

Description: Purchase to which the item belongs.

Column: material_id

Data Type: INT

Constraints: NOT NULL, FOREIGN KEY.

Description: Material purchased.

Column: quantity

Data Type: INT

Constraints: NOT NULL, CHECK (quantity > 0).

Description: Number of units purchased.

Column: unit_price

Data Type: DECIMAL(10,2)

Constraints: NOT NULL.

Description: Price paid per unit for the purchase item.

7.7. INVENTORY TABLE

Column: material_id

Data Type: INT

Constraints: PRIMARY KEY, FOREIGN KEY, ON DELETE CASCADE.

Description: Identifier of the material whose stock is maintained.

Column: quantity

Data Type: INT

Constraints: NOT NULL, DEFAULT 0, CHECK (quantity >= 0).

Description: Current quantity available in inventory.

Column: last_updated

Data Type: TIMESTAMP

Constraints: Automatically maintained.

Description: Date and time of the latest inventory update.

7.8. MATERIAL_USAGE TABLE

Column: usage_id

Data Type: INT

Constraints: PRIMARY KEY, AUTO_INCREMENT.

Description: Unique identifier for each material allocation record.

Column: site_id

Data Type: INT

Constraints: NOT NULL, FOREIGN KEY.

Description: Construction site receiving the material.

Column: material_id

Data Type: INT

Constraints: NOT NULL, FOREIGN KEY.

Description: Material allocated to the site.

Column: emp_id

Data Type: INT

Constraints: NOT NULL, FOREIGN KEY.

Description: Employee who issued the material.

Column: quantity

Data Type: INT

Constraints: NOT NULL, CHECK (quantity > 0).

Description: Quantity of material allocated.

Column: usage_date

Data Type: DATE

Constraints: NOT NULL, DEFAULT current date.

Description: Date on which the material was allocated.

8. DATABASE OBJECTS

BuildStock uses database functions, views, triggers, stored procedures, and transactions to automate calculations and enforce business rules.

8.1. STORED FUNCTION

Function Name: fn_stock_status

Parameters: qty INT, reorder INT.

Purpose: Determines the current stock status of a material.

Logic:

If the quantity is zero or less, the function returns "Out of stock".

If the quantity is greater than zero but less than or equal to the reorder level, it returns "Low stock".

If the quantity is greater than the reorder level, it returns "In stock".

The function provides a consistent way to determine material status in inventory reports.

8.2. DATABASE VIEWS

View Name: v_inventory

Purpose: Displays each material with its available quantity, minimum stock level, stock status, and stock value. The status is determined using fn_stock_status.

View Name: v_purchase_summary

Purpose: Displays purchase details along with supplier names, employee names, and total purchase amounts.

View Name: v_site_usage

Purpose: Summarizes the quantities and costs of materials allocated to each construction site.

8.3. DATABASE TRIGGERS

Trigger Name: trg_material_inventory

Event: AFTER INSERT ON Materials.

Purpose: Automatically creates an Inventory record with an initial quantity of zero whenever a new material is inserted.

Trigger Name: trg_purchase_stock

Event: AFTER INSERT ON Purchase_Items.

Purpose: Increases the corresponding inventory quantity and adds the purchase item's value to the associated purchase total.

Trigger Name: trg_usage_check

Event: BEFORE INSERT ON Material_Usage.

Purpose: Checks whether sufficient stock is available before a material allocation is recorded. If the available stock is insufficient, the trigger raises an error with the message "Insufficient stock for this material".

Trigger Name: trg_usage_stock

Event: AFTER INSERT ON Material_Usage.

Purpose: Reduces the inventory quantity by the allocated amount after a valid material usage record is inserted.

8.4. STORED PROCEDURE

Procedure Name: sp_record_usage

Parameters: p_site, p_material, p_emp, p_qty.

Purpose: Records the allocation of material to a construction site within a transaction.

Processing:

1. The procedure starts a transaction.
2. It inserts a record into Material_Usage.
3. The database trigger checks stock availability and updates the inventory quantity.
4. If the operation succeeds, the transaction is committed.

5. If an error occurs, the transaction is rolled back and the error is raised again.

The application calls this procedure whenever a user allocates materials to a site.

8.5. TRANSACTION MANAGEMENT

Purchase Transactions:

The application starts a transaction before inserting a purchase. It inserts the purchase record and all associated purchase items. Database triggers update stock quantities and the purchase total. If all operations succeed, the transaction is committed. If an operation fails, the transaction is rolled back.

Material Allocation Transactions:

The sp_record_usage procedure records the allocation and invokes the database trigger logic. If stock is insufficient or another database error occurs, the transaction is rolled back.

Transactions help prevent incomplete purchases and invalid material allocations from being retained in the database.

9. HOW THE APPLICATION WORKS

9.1. APPLICATION PAGES AND API ENDPOINTS

Dashboard

Endpoint: /api/dashboard and /api/inventory

Purpose: Displays important counts, inventory information, and low-stock warnings.

Materials

Purpose: Allows users to add, view, edit, delete, and search material records through generic CRUD routes.

Suppliers

Purpose: Allows users to manage supplier details through generic CRUD routes.

Employees

Purpose: Allows users to maintain employee records through generic CRUD routes.

Sites

Purpose: Allows users to maintain construction site details through generic CRUD routes.

Purchase Entry

Endpoint: /api/purchases

Purpose: Records purchases and their associated material items. The backend manages the transaction, while database triggers update inventory quantities and purchase totals.

Inventory

Endpoint: /api/inventory

Purpose: Displays current stock quantities, stock status, and stock value using the v_inventory view.

Low Stock

Endpoint: /api/low-stock

Purpose: Lists materials whose stock quantities are at or below their minimum stock levels.

Material Allocation

Endpoint: /api/usage and /api/inventory

Purpose: Records material allocations and retrieves current inventory information.

Usage by Site

Endpoint: /api/site-usage/:id

Purpose: Displays material usage and associated costs for a selected construction site.

Reports

Endpoint: /api/reports

Purpose: Displays purchase and material usage summaries.

9.2. PURCHASE RECORDING WORKFLOW

Step 1: The user selects a supplier and employee, enters purchase details, adds material items, and clicks Save.

Step 2: The browser sends a POST request to /api/purchases.

Step 3: The server starts a database transaction.

Step 4: The server inserts the main purchase record into Purchases.

Step 5: The server inserts each material item into Purchase_Items.

Step 6: The trg_purchase_stock trigger increases stock quantities and updates the purchase total.

Step 7: The server commits the transaction if all operations succeed.

Step 8: The browser displays a success message after receiving the successful response.

If an operation fails, the server rolls back the transaction so that incomplete purchase data is not retained.

9.3. MATERIAL ALLOCATION WORKFLOW

Step 1: The user selects the construction site, material, employee, and quantity.

Step 2: The interface displays the available stock and may prevent quantities greater than the available amount.

Step 3: The browser sends a POST request to /api/usage.

Step 4: The server calls sp_record_usage.

Step 5: The trg_usage_check trigger verifies stock availability.

Step 6: If sufficient stock exists, a usage record is inserted and trg_usage_stock reduces the available quantity.

Step 7: If the operation fails, the database raises an error and the transaction is rolled back.

Step 8: The browser displays the appropriate success or error message.

The frontend check improves usability, while the database trigger provides an additional safeguard against invalid allocations.

9.4. LOW-STOCK CALCULATION

A material is classified as low stock when its available quantity is less than or equal to its reorder level.